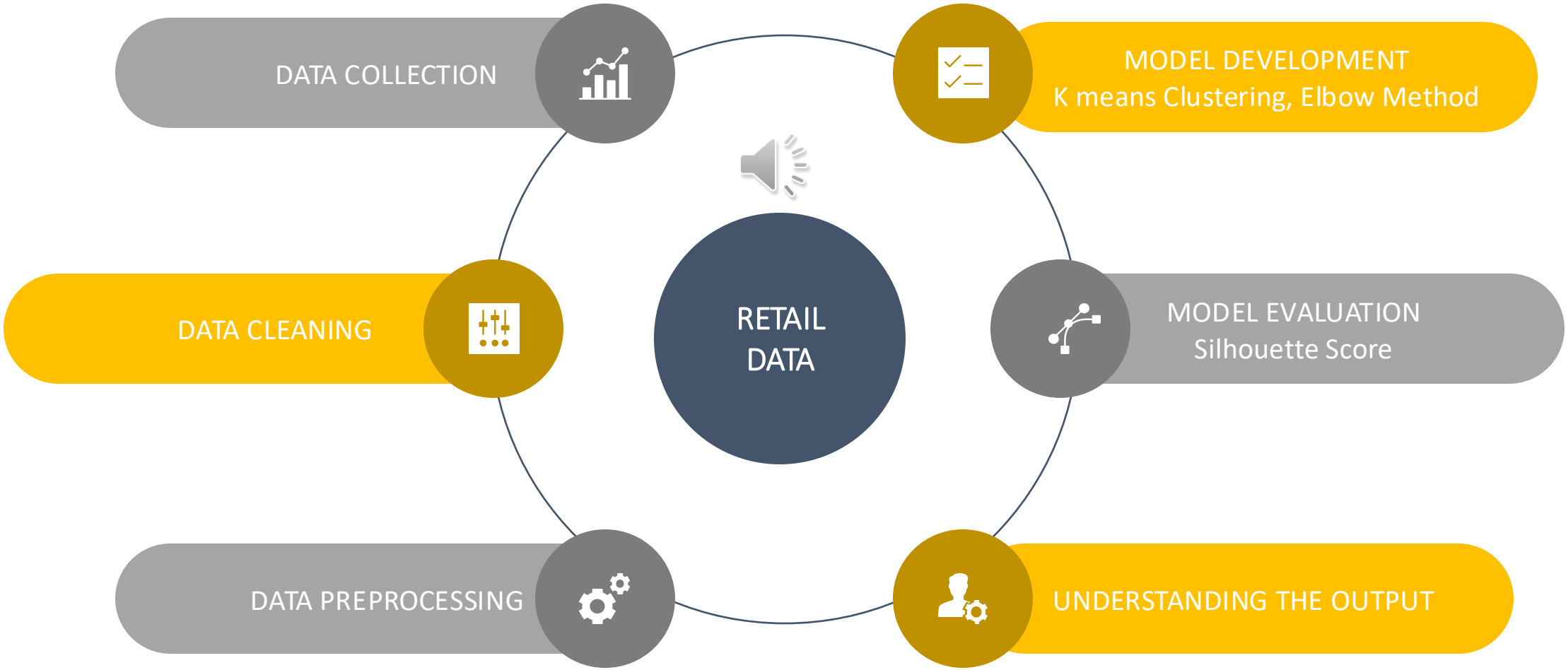
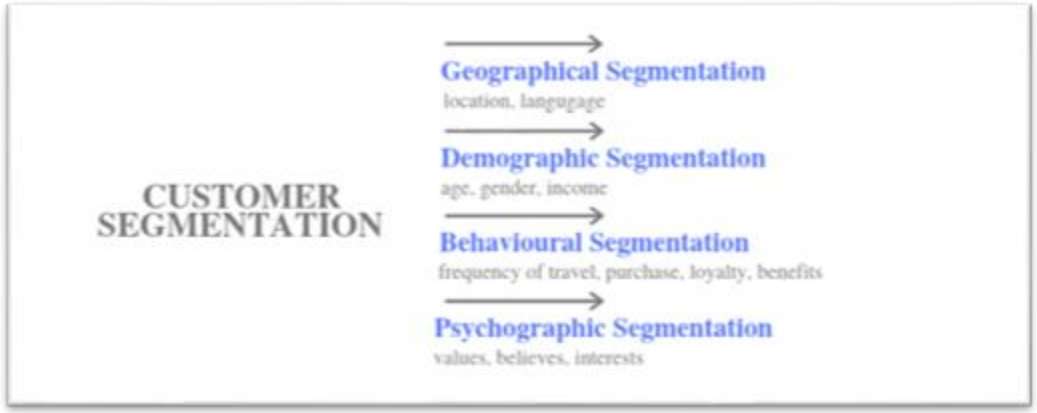
Retail Data grouping using K means clustering

Venkata Gayathri Peri

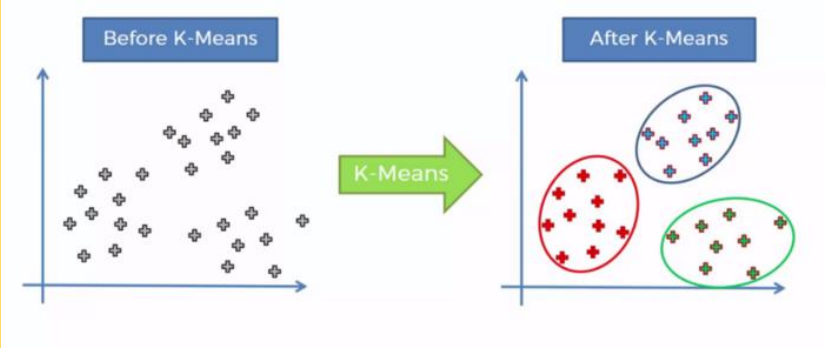
What was done?
The Project Plan



K means Clustering



- Unsupervised machine learning algorithm
- Simplest way to divide data into group of clusters
- Used for pattern recognition ,datamining and other applications of ML



Retail Data Analysis

Data Collection

- Data taken from online datasets resource git hub. dataset
- Data was loaded into data bricks spark data frame

Data Cleaning & Preprocessing

- Revenue was calculated and added as a new column
- Nulls were checked and removed
- Orders with products in it are filtered

Machine learning model development

- Preprocessed data is downloaded and converted to pandas' data frame
- Necessary libraries used
- Elbow plot – Optimal K value achieved

Model Evaluation

- Clusters are formed
- Silhouette score is calculated which tells how well datapoints are clustered

Final Output

- Clusters are observed and some outliers in data are still present and to be worked on in the second phase
- Optimal K and silhouette score achieved
- Pattern recognition for further prediction analysis is achieved

Project Code

databricks

Project Data Model (Python)

Import Notebook

```
from pyspark.sql import SparkSession

# Create a SparkSession
spark = SparkSession.builder.appName("MyKmeans").getOrCreate()

# Now you can use Spark and PySpark APIs
```

```
# Load the CSV file into a DataFrame
df = spark.read.csv("/FileStore/tables/MyOnlineRetail.csv", header=True, inferSchema=True)

# Show the DataFrame
df.show()
```

```
+---------+---------+--------------------+--------+--------------+---------+----------+--------------+-------+
|InvoiceNo|StockCode|         Description|Quantity|   InvoiceDate|UnitPrice|CustomerID|       Country|revenue|
+---------+---------+--------------------+--------+--------------+---------+----------+--------------+-------+
|   536365|   85123A|WHITE HANGING HEA...|       6|1/12/2010 8:26|     2.55|     17850|United Kingdom|   15.3|
|   536365|    71053| WHITE METAL LANTERN|       6|1/12/2010 8:26|     3.39|     17850|United Kingdom|  20.34|
|   536365|   84406B|CREAM CUPID HEART...|       8|1/12/2010 8:26|     2.75|     17850|United Kingdom|   22.0|
|   536365|   84029G|KNITTED UNION FLA...|       6|1/12/2010 8:26|     3.39|     17850|United Kingdom|  20.34|
|   536365|   84029E|RED WOOLLY HOTTIE...|       6|1/12/2010 8:26|     3.39|     17850|United Kingdom|  20.34|
|   536365|    22752|SET 7 BABUSHKA NE...|       2|1/12/2010 8:26|     7.65|     17850|United Kingdom|   15.3|
|   536365|    21730|GLASS STAR FROSTE...|       6|1/12/2010 8:26|     4.25|     17850|United Kingdom|   25.5|
|   536366|    22633|HAND WARMER UNION...|       6|1/12/2010 8:28|     1.85|     17850|United Kingdom|   11.1|
|   536366|    22632|HAND WARMER RED P...|       6|1/12/2010 8:28|     1.85|     17850|United Kingdom|   11.1|
|   536367|    84879|ASSORTED COLOUR B...|      32|1/12/2010 8:34|     1.69|     13047|United Kingdom|  54.08|
|   536367|    22745|POPPY'S PLAYHOUSE...|       6|1/12/2010 8:34|      2.1|     13047|United Kingdom|   12.6|
|   536367|    22748|POPPY'S PLAYHOUSE...|       6|1/12/2010 8:34|      2.1|     13047|United Kingdom|   12.6|
|   536367|    22749|FELTCRAFT PRINCES...|       8|1/12/2010 8:34|     3.75|     13047|United Kingdom|   30.0|
|   536367|    22310|IVORY KNITTED MUG...|       6|1/12/2010 8:34|     1.65|     13047|United Kingdom|    9.9|
|   536367|    84969|BOX OF 6 ASSORTED...|       6|1/12/2010 8:34|     4.25|     13047|United Kingdom|   25.5|
|   536367|    22623|BOX OF VINTAGE JI...|       3|1/12/2010 8:34|     4.95|     13047|United Kingdom|  14.85|
|   536367|    22622|BOX OF VINTAGE AL...|       2|1/12/2010 8:34|     9.95|     13047|United Kingdom|   19.9|
|   536367|    21754|HOME BUILDING BLO...|       3|1/12/2010 8:34|     5.95|     13047|United Kingdom|  17.85|
```

```
#cache the dataframe
df.cache()
df.show()
```

Project Code

databricks

Project Data Model (Python)

Import Notebook

```
#checking datatypes of each field
df.printSchema()
```

```
root
 |-- InvoiceNo: string (nullable = true)
 |-- StockCode: string (nullable = true)
 |-- Description: string (nullable = true)
 |-- Quantity: integer (nullable = true)
 |-- InvoiceDate: string (nullable = true)
 |-- UnitPrice: double (nullable = true)
 |-- CustomerID: integer (nullable = true)
 |-- Country: string (nullable = true)
 |-- revenue: double (nullable = true)
```

```
#check for null data
from pyspark.sql.functions import count, when, isnull

# Check for null values in the DataFrame
null_counts = df.select([count(when(isnull(c), c)).alias(c) for c in df.columns]).collect()[0]

# Print the null counts for each column
for column in df.columns:
    print(f"Column {column} has {null_counts[column]} null values")
```

```
Column InvoiceNo has 0 null values
Column StockCode has 0 null values
Column Description has 1454 null values
Column Quantity has 0 null values
Column InvoiceDate has 0 null values
Column UnitPrice has 0 null values
Column CustomerID has 135080 null values
Column Country has 0 null values
Column revenue has 0 null values
```

```
df.count()
```

```
Out[7]: 541909
```

Project Code

```
# Drop null records of the "col1" column in place
df=df.dropna(subset=["CustomerID"])

# Show the resulting DataFrame

df.count()


Out[9]: 406829
```

```
#nulls have been removed and count of non null records displayed and now let us view the dataframe
df.show()
```

```
+---------+---------+--------------------+--------+--------------+---------+----------+--------------+-------+
|InvoiceNo|StockCode|         Description|Quantity|   InvoiceDate|UnitPrice|CustomerID|       Country|revenue|
+---------+---------+--------------------+--------+--------------+---------+----------+--------------+-------+
|   536365|   85123A|WHITE HANGING HEA...|       6|1/12/2010 8:26|     2.55|     17850|United Kingdom|   15.3|
|   536365|    71053| WHITE METAL LANTERN|       6|1/12/2010 8:26|     3.39|     17850|United Kingdom|  20.34|
|   536365|   84406B|CREAM CUPID HEART...|       8|1/12/2010 8:26|     2.75|     17850|United Kingdom|   22.0|
|   536365|   84029G|KNITTED UNION FLA...|       6|1/12/2010 8:26|     3.39|     17850|United Kingdom|  20.34|
|   536365|   84029E|RED WOOLLY HOTTIE...|       6|1/12/2010 8:26|     3.39|     17850|United Kingdom|  20.34|
|   536365|    22752|SET 7 BABUSHKA NE...|       2|1/12/2010 8:26|     7.65|     17850|United Kingdom|   15.3|
|   536365|    21730|GLASS STAR FROSTE...|       6|1/12/2010 8:26|     4.25|     17850|United Kingdom|   25.5|
|   536366|    22633|HAND WARMER UNION...|       6|1/12/2010 8:28|     1.85|     17850|United Kingdom|   11.1|
|   536366|    22632|HAND WARMER RED P...|       6|1/12/2010 8:28|     1.85|     17850|United Kingdom|   11.1|
|   536367|    84879|ASSORTED COLOUR B...|      32|1/12/2010 8:34|     1.69|     13047|United Kingdom|  54.08|
|   536367|    22745|POPPY'S PLAYHOUSE...|       6|1/12/2010 8:34|      2.1|     13047|United Kingdom|   12.6|
|   536367|    22748|POPPY'S PLAYHOUSE...|       6|1/12/2010 8:34|      2.1|     13047|United Kingdom|   12.6|
|   536367|    22749|FELTCRAFT PRINCES...|       8|1/12/2010 8:34|     3.75|     13047|United Kingdom|   30.0|
|   536367|    22310|IVORY KNITTED MUG...|       6|1/12/2010 8:34|     1.65|     13047|United Kingdom|    9.9|
|   536367|    84969|BOX OF 6 ASSORTED...|       6|1/12/2010 8:34|     4.25|     13047|United Kingdom|   25.5|
|   536367|    22623|BOX OF VINTAGE JI...|       3|1/12/2010 8:34|     4.95|     13047|United Kingdom|  14.85|
|   536367|    22622|BOX OF VINTAGE AL...|       2|1/12/2010 8:34|     9.95|     13047|United Kingdom|   19.9|
|   536367|    21754|HOME BUILDING BLO...|       3|1/12/2010 8:34|     5.95|     13047|United Kingdom|  17.85|
```

```
display(df)
df.count()
```

Project Code

```
display(df)
df.count()
```

Table

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	revenue
1	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	1/12/2010 8:26	2.55	17850	United Kingdom	15.3
2	536365	71053	WHITE METAL LANTERN	6	1/12/2010 8:26	3.39	17850	United Kingdom	20.34
3	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	1/12/2010 8:26	2.75	17850	United Kingdom	22
4	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	1/12/2010 8:26	3.39	17850	United Kingdom	20.34
5	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	1/12/2010 8:26	3.39	17850	United Kingdom	20.34
6	536365	22752	SET 7 BABUSHKA NESTING BOXES	2	1/12/2010 8:26	7.65	17850	United Kingdom	15.3
7	536365	21730	GLASS STAR FROSTED T-LIGHT HOLDER	6	1/12/2010 8:26	4.25	17850	United Kingdom	25.5

10,000 rows | Truncated data

```
Out[16]: 406829
```

```
from pyspark.sql.functions import col
df = df.filter(col("Quantity") >= 0)
df.count()
display(df)
```

Table

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	revenue
1	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	1/12/2010 8:26	2.55	17850	United Kingdom	15.3
2	536365	71053	WHITE METAL LANTERN	6	1/12/2010 8:26	3.39	17850	United Kingdom	20.34
3	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	1/12/2010 8:26	2.75	17850	United Kingdom	22
4	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	1/12/2010 8:26	3.39	17850	United Kingdom	20.34
5	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	1/12/2010 8:26	3.39	17850	United Kingdom	20.34
6	536365	22752	SET 7 BABUSHKA NESTING BOXES	2	1/12/2010 8:26	7.65	17850	United Kingdom	15.3
7	536365	21730	GLASS STAR FROSTED T-LIGHT HOLDER	6	1/12/2010 8:26	4.25	17850	United Kingdom	25.5

10,000 rows | Truncated data

Project Code

databricks

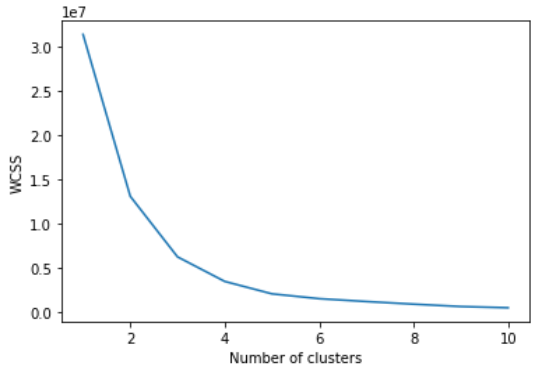
Project Data Model (Python)

Import Notebook

```
df.count()
```

```
Out[21]: 397924
```

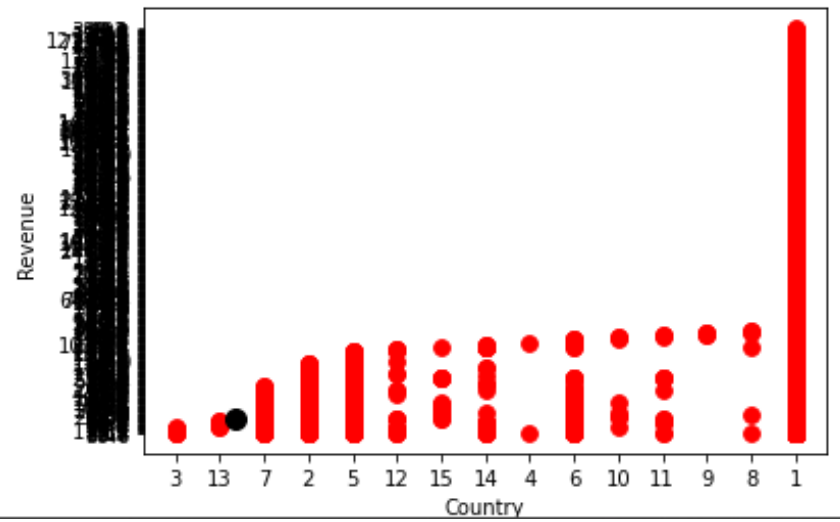
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import sklearn
# Read the CSV file using Spark
df1 = spark.read.format("csv").option("header", "true").load("dbfs:/FileStore/shared_uploads/venkatagayathrip1498@gmail.com/preprocessed_dataset-1.csv")
# Convert the Spark DataFrame to a pandas DataFrame
pdf1 = df1.toPandas()
# Extract the values for clustering
X = pdf1.iloc[:, [8,9]].values
from sklearn.cluster import KMeans
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, init='k-means++', random_state=42)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)
plt.plot(range(1, 11), wcss)
plt.xlabel('Number of clusters')
plt.ylabel('WCSS')
plt.show()
```



Project Code

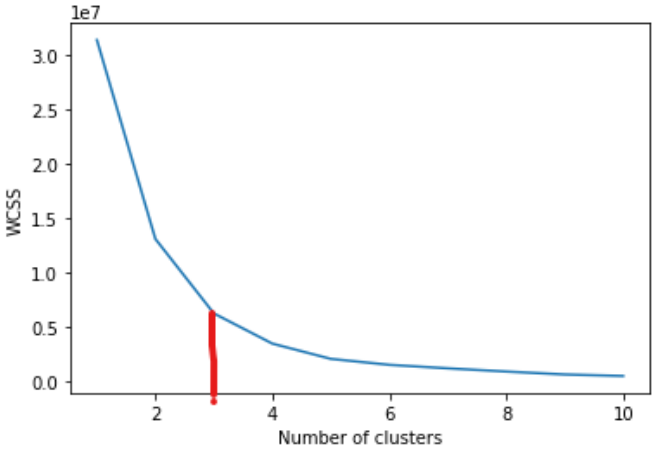
```
kmeans = KMeans(n_clusters = 3, init = "k-means++", random_state = 42)
y_kmeans = kmeans.fit_predict(X)
from sklearn.metrics import silhouette_score
# Compute silhouette score
silhouette_avg = silhouette_score(X, y_kmeans)
print('Silhouette Score:', silhouette_avg)

Silhouette Score: 0.9199717159055351
```

```
plt.scatter(X[y_kmeans == 0, 0], X[y_kmeans == 0, 1], s = 60, c = 'red', label = 'Cluster1')
plt.scatter(X[y_kmeans == 1, 0], X[y_kmeans == 1, 1], s = 60, c = 'blue', label = 'Cluster2')
plt.scatter(X[y_kmeans == 2, 0], X[y_kmeans == 2, 1], s = 60, c = 'green', label = 'Cluster3')
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], s = 100, c = 'black', label = 'Centroids')
plt.xlabel('Country')
plt.ylabel('Revenue')
plt.show()
```

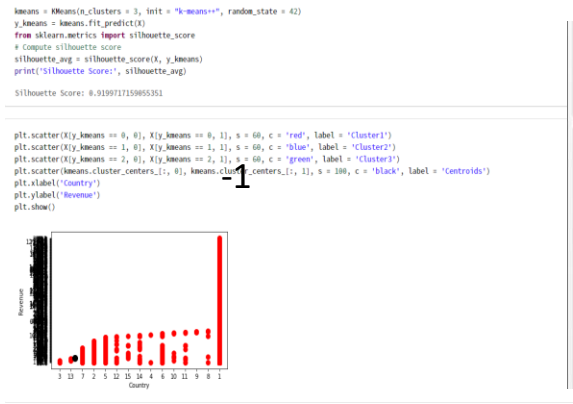


Model Evaluation

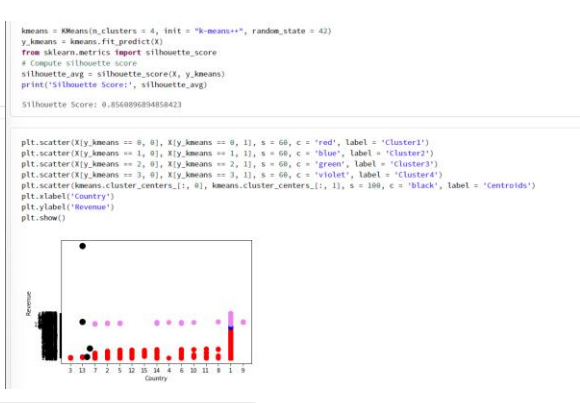
Elbow Plot



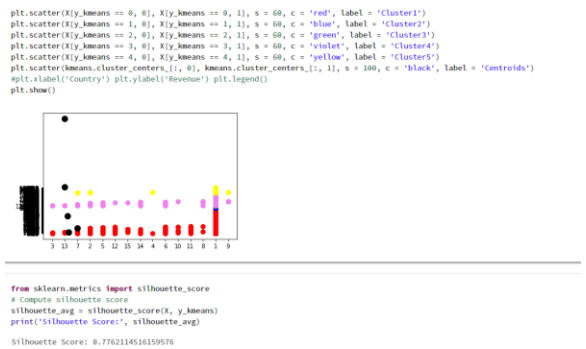
K=3
Score=0.91



K=4
Score=0.85



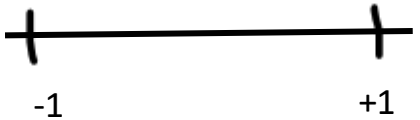
K=5
Score=0.77



For k=3 the highest silhouette score is achieved

K= 4 and k=5 comparison shows how silhouette score is low compared to the optimal K achieved through elbow method.

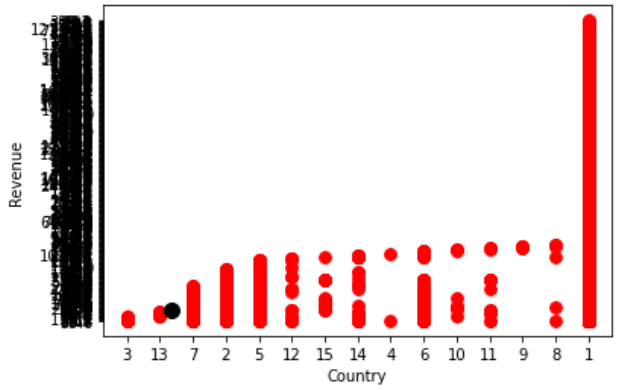
Silhouette score range



Final Output

```
kmeans = KMeans(n_clusters = 3, init = "k-means++", random_state = 42)
y_kmeans = kmeans.fit_predict(X)
from sklearn.metrics import silhouette_score
# Compute silhouette score
silhouette_avg = silhouette_score(X, y_kmeans)
print('Silhouette Score:', silhouette_avg)

Silhouette Score: 0.9199717159055351
```

```
plt.scatter(X[y_kmeans == 0, 0], X[y_kmeans == 0, 1], s = 60, c = 'red', label = 'Cluster1')
plt.scatter(X[y_kmeans == 1, 0], X[y_kmeans == 1, 1], s = 60, c = 'blue', label = 'Cluster2')
plt.scatter(X[y_kmeans == 2, 0], X[y_kmeans == 2, 1], s = 60, c = 'green', label = 'Cluster3')
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], s = 100, c = 'black', label = 'Centroids')
plt.xlabel('Country')
plt.ylabel('Revenue')
plt.show()
```



Project Outcomes

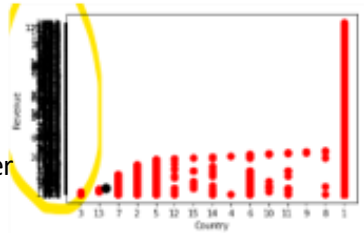
Clusters are formed such that one cluster will have countries with high revenue and other cluster will have countries with low revenue .

Outliers in data / data imbalances to be handled for better cluster visualization and to go ahead supervised learning for predictions

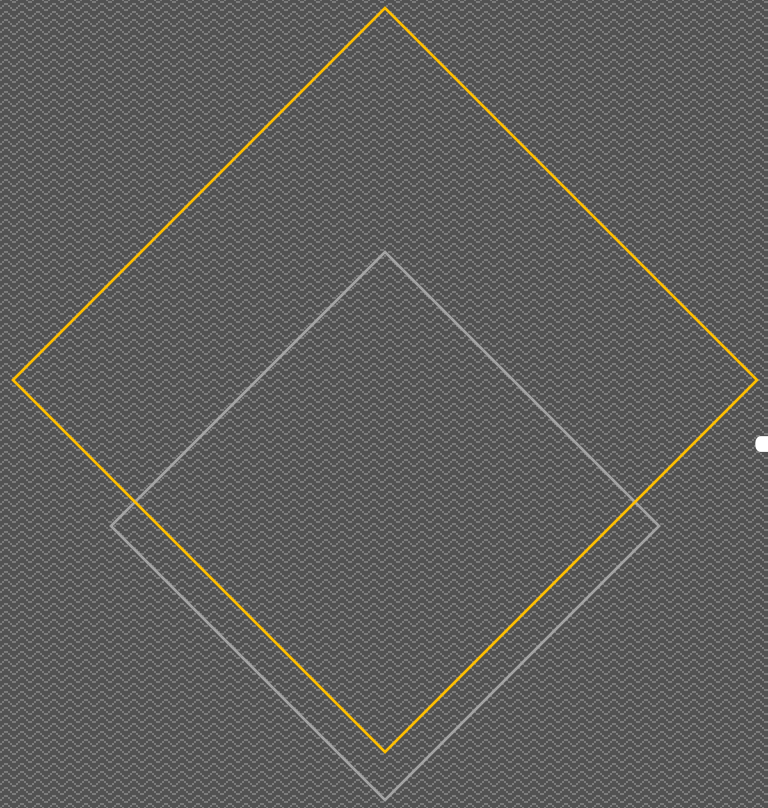
Customer segmentation achieved based on geographic/demographic data and revenue generated

Patterns analyzed through this clustering

Data outliers to be removed for predictive analysis



Outliers could be the reason for the scatter plot's ambiguity, and this needs clear data inspection



Thank You